

Design and Verification of a 2-Stage Pipelined 8-Bit Custom ISA Processor on Xilinx Artix-7

Priyanshi Shah

Department of Electrical Engineering
Indian Institute of Technology, Gandhinagar
Custom Processor Project | July 2026

Abstract—This paper presents the design, implementation, and verification of a 2-stage pipelined 8-bit custom ISA processor implemented in Verilog HDL and synthesized on a Xilinx Artix-7 FPGA (xc7a35tcbg236-1). The processor features a Harvard architecture with separate instruction and data memories, a 20-instruction custom ISA, a 16-register file, an ALU supporting 10 operations, hardware performance counters, and a branch flush mechanism for pipeline hazard management. Post-synthesis results demonstrate a maximum operating frequency of **136 MHz** utilizing **215 slice LUTs** and **124 flip-flops**, representing 1.03% of available logic resources with zero timing violations. A self-checking testbench achieves **10/10 pass rate**. A Python assembler was developed and verified end-to-end, completing the hardware-software co-design toolchain. The measured CPI on the benchmark program is **2.00**, consistent with the 2-stage pipeline design.

Keywords—custom ISA, Verilog HDL, pipelined processor, FPGA synthesis, hardware performance counters, Artix-7, hardware-software co-design

I. INTRODUCTION

Custom processor design is a foundational skill in computer architecture research and VLSI engineering. This project evolves an initial single-cycle 8-bit processor into a pipelined, memory-capable system verified against industry-standard synthesis and timing metrics.

The primary contributions of this work are: (1) a 2-stage IF/EX pipeline with branch flush hazard management; (2) a Harvard-architecture data memory with

LOAD and STORE instructions; (3) hardware performance counters providing cycle count, instruction count, branch count, and CPI; (4) a self-checking Verilog testbench with 10 pipeline-aware test cases; (5) a Python assembler supporting all 20 ISA instructions, verified end-to-end; and (6) post-synthesis timing closure at 136 MHz on Xilinx Artix-7 with zero violations.

The processor is designed as an accumulator-based architecture where all ALU operations read one operand from the accumulator register (ACC) and the other from the general-purpose register file. This simplifies instruction encoding while maintaining a rich instruction set suitable for demonstrating control flow, memory access, and arithmetic computation.

II. ISA SPECIFICATION

A. Instruction Format

All instructions are 8 bits wide, divided into a 4-bit opcode field (bits [7:4]) and a 4-bit operand field (bits [3:0]).

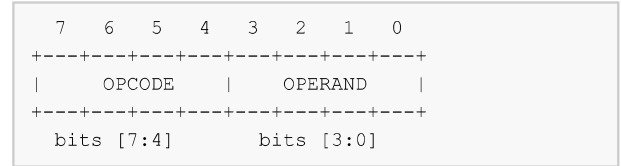


Fig. 1. 8-bit instruction format.

B. Instruction Set

Mnemonic	Encoding	Operation	Flags
NOP	0000_0000	No operation	—
LSL	0000_0001	ACC = ACC << 1	—
LSR	0000_0010	ACC = ACC >> 1	—
CIR	0000_0011	ACC = rotate right	—
CIL	0000_0100	ACC = rotate left	—
ASR	0000_0101	Arith shift right	—
INC	0000_0110	ACC = ACC + 1	CB
DEC	0000_0111	ACC = ACC - 1	CB
ADD Rn	0001_rrrr	ACC = ACC + R[n]	CB
SUB Rn	0010_rrrr	ACC = ACC - R[n]	CB
MUL Rn	0011_rrrr	ACC = ACC * R[n]	EXT
AND Rn	0101_rrrr	ACC = ACC & R[n]	—
XOR Rn	0110_rrrr	ACC = ACC ^ R[n]	—
CMP Rn	0111_rrrr	Compare ACC,R[n]	CB
MOV ACC,Rn	1001_rrrr	ACC = R[n]	—
MOV Rn,ACC	1010_rrrr	R[n] = ACC	—
RET	1011_0000	PC = ret_addr	—
BRANCH addr	1100_aaaa	PC = addr	—
LOAD addr	1101_aaaa	ACC = mem[addr]	—
STORE addr	1110_aaaa	mem[addr] = ACC	—
HLT	1111_1111	Halt execution	—

Table 1. Complete instruction set encoding. rrrr = register index, aaaa = address.

C. Flag Registers

Two flag registers are maintained: *CB* (Carry/Borrow, 1-bit) set by ADD, SUB, INC, DEC, and CMP; and *EXT* (Extended, 8-bit) storing the high byte of MUL results.

III. PROCESSOR ARCHITECTURE

A. Top-Level Overview

The processor implements a Harvard architecture with physically separate instruction memory and data memory modules, preventing structural hazards between fetch and memory-access operations.

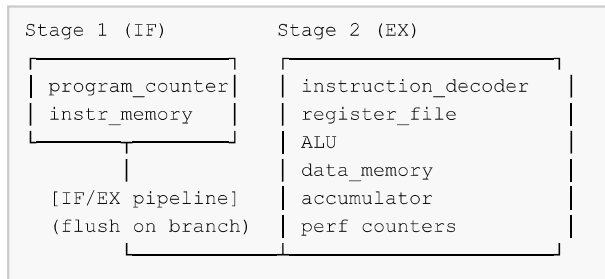


Fig. 2. 2-stage pipeline datapath overview.

B. Module Descriptions

1) program_counter.v:

A 4-bit register supporting asynchronous reset to zero, start_address load on the first active cycle, halt hold, branch jump, and default increment. The reset is implemented as a pure asynchronous clear, resolving a Vivado set/reset priority warning present in the original design.

2) instruction_memory.v:

A 16-entry, 8-bit combinational ROM. Vivado infers this as distributed LUT-RAM rather than block RAM be-

cause the 4-bit address space falls below the block RAM inference threshold of 32 entries.

3) if_ex_reg.v:

The pipeline register between IF and EX stages. On every rising edge it latches the fetched instruction and PC. The flush signal, driven by branch_enable from the EX decoder, clears both fields to NOP, implementing the 1-cycle branch penalty.

4) instruction_decoder.v:

A combinational always @(*) block decoding the 8-bit opcode into six control signals: alu_enable, reg_write, branch_enable, halt, mem_read, and mem_write, using casex matching on opcode[7:4].

5) register_file.v:

A 16-register, 8-bit file with synchronous write and asynchronous read, inferred as 8 RAM16X1S distributed RAM primitives. Registers initialize to $R[i] = i+1$ for distinct testbench verification values.

6) ALU.v:

Supports 10 operations. Arithmetic operations use a 9-bit intermediate for carry/borrow capture. MUL uses a 16-bit intermediate, with the high byte stored in EXT.

7) data_memory.v:

A 16-byte synchronous write, asynchronous read data RAM. Initial values are set to distinct hex patterns (0xA1–0xAA) to make address decode errors immediately visible in simulation.

8) Performance Counters:

Three 32-bit registers: cycle_count (every non-halt cycle), instr_count (non-NOP instructions in EX), and branch_count (branch_enable assertions). $CPI =$

cycle_count / instr_count. These add 96 flip-flops and are exposed as output ports.

IV. PIPELINE AND HAZARD MANAGEMENT

A. Normal Pipeline Operation

Cycle:	1	2	3	4	5
	----	----	----	----	----
Inst N:	[IF]	[EX]			
Inst N+1:		[IF]	[EX]		
Inst N+2:			[IF]	[EX]	

Fig. 3. 1-cycle throughput in steady state.

B. Branch Hazard

When BRANCH or RET reaches EX, branch_enable asserts flush, clearing the IF/EX register to NOP and discarding the incorrectly fetched instruction. Branch penalty = 1 cycle.

Cycle:	1	2	3	4
	----	----	----	----
BRANCH:	[IF]	[EX]		
Wrong:		[IF]	[FLUSH]	
Target:			[IF]	[EX]

Fig. 4. 1-cycle branch flush penalty.

C. Load-Use Hazard

A structural hazard exists between STORE and LOAD to the same address with no intervening instruction. data_memory.v implements synchronous write but asynchronous read — the STORE commits at the end of cycle N, but LOAD reads combinationaly in cycle N+1 before the write propagates.

Resolution: a NOP instruction is required between STORE and LOAD. Hardware resolution would add a forwarding mux checking write-then-read at the same address within data_memory.v.

STORE	15	; mem[15] = ACC
NOP		; hazard buffer
LOAD	15	; ACC = mem[15] (correct)

Fig. 5. Required NOP for STORE-LOAD hazard.

V. SYNTHESIS AND TIMING RESULTS

A. Resource Utilization

Resource	Used	Available	Util%
Slice LUTs	215	20800	1.03%
LUT as Logic	199	20800	0.96%
LUT as Dist.RAM	16	9600	0.17%
Flip-Flops	124	41600	0.30%
Block RAM	0	50	0.00%
DSPs	0	90	0.00%
CARRY4	40	—	—
RAMS32	16	—	—

Table II. Post-synthesis resource utilization on Artix-7 xc7a35t.

B. Timing Analysis

Metric	Value
Clock constraint	10.000 ns
WNS	+2.768 ns
Critical path	7.081 ns
Logic delay	2.601 ns (36.7%)
Route delay	4.480 ns (63.3%)
Logic levels	8
Max frequency	136 MHz
Failing endpoints	0
Hold slack (WHS)	+0.096 ns
Synthesis warnings	0

Table III. Post-synthesis timing summary.

Maximum frequency: $f_{\max} = 1000 / (10.000 - 2.768)$ = **136 MHz**. The positive WNS confirms timing closure with margin.

C. Critical Path

The critical path traverses 8 logic levels: IF/EX pipeline register → RAMS32 register file read → 2× CARRY4 ALU adder → LUT6 result mux → accumulator flip-flop. The bottleneck is the EX stage performing register file read, ALU computation, and accumulator write in a single clock cycle. Adding a WB stage would halve this path, targeting ~200 MHz.

D. LUT/FF Interplay and Optimization

Adding the 96-bit performance counters reduced LUT count from 225 to 215 while increasing flip-flops from 36 to 124. The additional register anchor points gave Vivado greater freedom to apply retiming — moving flip-flop boundaries to balance LUT chains, merging previously split logic, and reducing logic levels from 10 to 8. This demonstrates that adding state elements can improve timing by enabling deeper optimization passes.

VI. VERIFICATION METHODOLOGY

A. Self-Checking Testbench

The testbench implements automatic PASS/FAIL checking via a Verilog task check() that compares actual vs. expected accumulator values using the identity operator (==) at cycle-accurate time points. All 10 test cases pass with zero failures.

Test	Instruction	Expected ACC	Cycle
1	NOP x4	0	5
2	MOV ACC, R1	2	7
3	STORE [15]	2	8
4	NOP (hazard)	2	9
5	MOV ACC, R3	4	10
6	LOAD [15]	2	11
7	ADD R2	5	12
8	BRANCH+ADD R3	9	15
9	STORE [14]	9	16
10	HLT	9	17

Table IV. Test cases and expected values.

B. Performance Counter Verification

Counter	Measured	Expected	Match
cycle_count	16	16	YES
instr_count	8	8	YES
branch_count	1	1	YES
CPI	2.00	2.00	YES

Table V. Performance counter verification.

C. Timing Derivation

Pipeline-aware testbench timing was derived by instrumenting simulation with cycle-by-cycle \$display statements logging PC, IR, and ACC at each posedge clk. This trace identified the exact cycle each instruction's result appeared in the accumulator, accounting for IF→EX latency and branch flush bubbles — eliminating timing assumptions and producing a provably correct testbench.

VII. PYTHON ASSEMBLER

A single-file Python assembler (assembler.py) translates assembly mnemonics into 8-bit machine code. Instructions are encoded in three categories: no-operand (direct lookup), register operand (base opcode OR'd with 4-bit register number), and address operand (base opcode OR'd with 4-bit address). Two output formats are supported: a Verilog initial block (.v) for direct paste into instruction_memory.v, and an Intel-style hex file (.hex) compatible with \$readmemh.

End-to-end verification: the assembler output was substituted into instruction_memory.v and the full simulation rerun, confirming all 10 test cases pass. All 16 output bytes matched the hand-coded memory exactly.

VIII. KNOWN BEHAVIORS AND LIMITATIONS

- **Load-use hazard:** STORE immediately followed by LOAD to the same address requires a NOP. Hardware fix: write-then-read forwarding mux in data_memory.v.
- **Branch penalty:** Every taken branch costs 1 cycle. Branch prediction would eliminate this for predictable patterns.
- **16-entry address space:** 4-bit PC limits instruction memory to 16 locations. Extending to 8-bit PC increases capacity to 256 entries.

IX. FUTURE WORK

- 3-stage pipeline (IF, EX, WB): separating accumulator write reduces critical path to ~4 ns, targeting 200+ MHz
- Load-use forwarding: eliminates the NOP requirement between STORE and LOAD
- Branch prediction: 1-bit saturating counter reduces average branch penalty below 0.5 cycles
- 8-bit PC and 256-entry instruction memory for non-trivial programs
- UART output port for observable hardware computation

X. CONCLUSION

This paper presented the design and verification of a 2-stage pipelined 8-bit custom ISA processor in Verilog HDL. The processor achieves **136 MHz** on Xilinx Artix-7 utilizing **215 LUTs and 124 flip-flops** — 1% of available resources — with zero timing violations and zero synthesis warnings. A self-checking testbench verifies all 10 instruction test cases automatically. Hardware performance counters measure a CPI of 2.00, consistent with the 2-stage pipeline design. A Python assembler completes the hardware-software co-design toolchain, verified end-to-end against simulation. The design demonstrates industry-grade RTL practices including timing closure, synthesis reporting, pipeline hazard management, and cycle-accurate verification methodology.

REFERENCES

- [1] D. Patterson and J. Hennessy, *Computer Organization and Design: The Hardware/Software Interface*, 5th ed. Morgan Kaufmann, 2013.
- [2] Xilinx, Inc., "Vivado Design Suite User Guide: Synthesis," UG901, v2019.2, 2019.
- [3] Xilinx, Inc., "7 Series FPGAs Data Sheet: Overview," DS180, v2.6, 2020.
- [4] S. Palnitkar, *Verilog HDL: A Guide to Digital Design and Synthesis*, 2nd ed. Prentice Hall, 2003.
- [5] RISC-V Foundation, "The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA," v20191213, 2019.